Lector SNMP de servidores Ubuntu

Monitoreo, histórico y gestión de servidores vía Docker. Explicación de la idea del proyecto.

1. ¿De qué trata el proyecto?

Es un sistema de monitoreo liviano. Cada cierto tiempo (por defecto, cada 5 minutos) consulta a un conjunto de servidores Ubuntu usando el protocolo **SNMP**, guarda las mediciones en una base de datos **MySQL** y las muestra en una **página web**: qué servidores están respondiendo, cuánta CPU, RAM y disco usan, hace cuánto tiempo están encendidos y cómo evolucionó todo eso en el tiempo.

Resuelve un problema típico de administración de sistemas: saber si los servidores están vivos y sanos sin entrar por SSH a cada uno, y conservar un histórico para detectar tendencias (por ejemplo, un disco que se va llenando de a poco o un servidor que cada tarde llega al 90% de CPU).

Todo el sistema se levanta con **Docker Compose** como tres servicios independientes dentro de una misma red: **worker**, **mysql** y **web**.

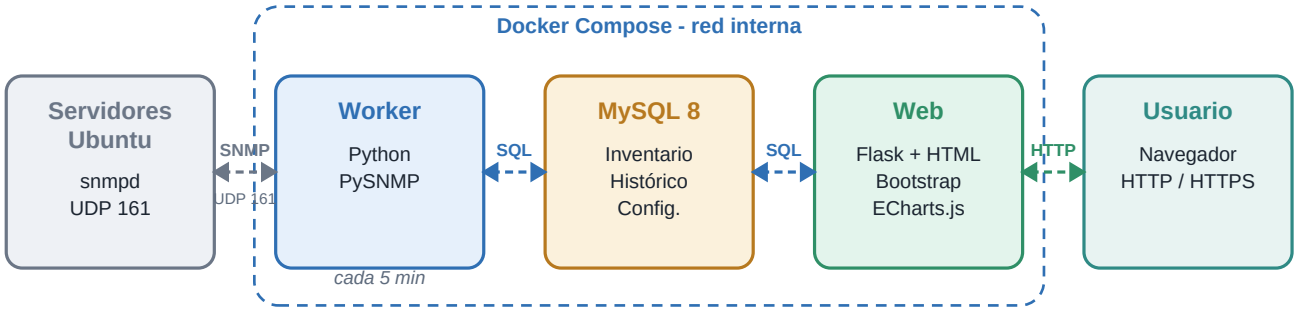
2. Qué es SNMP y qué se le pregunta a cada servidor

SNMP (Simple Network Management Protocol) es un protocolo estándar para consultar el estado de equipos de red y servidores. En cada Ubuntu corre un **agente** (`snmpd`) que escucha en el puerto UDP 161. Quien consulta (acá, el worker) envía una pregunta con una **community** (una especie de contraseña de solo lectura) y un **OID**, que es el identificador numérico del dato que quiere leer.

Métrica	Objeto (MIB)	OID	Cómo se usa
Uptime	hrSystemUptime.0	`1.3.6.1.2.1.25.1.1.0`	Centésimas de segundo desde el último arranque; se divide por 100.
CPU	ssCpuIdle.0	`1.3.6.1.4.1.2021.11.11.0`	Porcentaje de CPU ociosa. Uso = 100 - idle.
RAM	memTotalReal.0 y memAvailReal.0	`1.3.6.1.4.1.2021.4.5.0` `1.3.6.1.4.1.2021.4.6.0`	En KB. Uso % = (total - libre - buffers - caché) / total. Buffers y caché (memBuffer.0 y memCached.0) se restan porque son memoria reciclable.
Disco	dskPercent.1	`1.3.6.1.4.1.2021.9.1.9.1`	Porcentaje usado. Requiere una línea `disk /` en snmpd.conf.

Son los OIDs estándar de Net-SNMP. En Ubuntu, el snmpd por defecto solo escucha en localhost y expone la rama *system*, así que hay que ajustar snmpd.conf en cada servidor (el proyecto incluye el script `docs/setup-snmpd.sh`). Conviene validar los OIDs con `snmpwalk` o `snmpget`.

3. Arquitectura



Servicio	Qué hace
Worker Python + PySNMP	Lee de MySQL la lista de servidores habilitados, los consulta por SNMP, procesa CPU, RAM, disco y uptime, y guarda una fila nueva en `metrics`. Corre en bucle cada N minutos (N sale de la tabla de configuración).
MySQL 8	Guarda el inventario de servidores, el histórico de métricas y la configuración del monitoreo. Usa un volumen persistente para que los datos sobrevivan a reinicios del contenedor.
Web Flask + HTML	Interfaz para el usuario: alta, baja y modificación de servidores (ABM), estado de disponibilidad, gráficos históricos y consulta de mediciones. Usa Bootstrap y ECharts.js. No habla SNMP: solo lee y escribe en MySQL.

Idea de diseño clave: el worker y la web nunca se comunican entre sí; **MySQL es el punto de encuentro**. Si la web se cae, el worker sigue guardando mediciones; si el worker falla, la web sigue mostrando el histórico. Cada pieza se puede desarrollar y probar por separado, que es exactamente lo que vamos a hacer.

4. Flujo de funcionamiento

1. El usuario registra un servidor desde la web (hostname, IP y community).
2. El worker obtiene la IP de los servidores habilitados desde MySQL.
3. Consulta el agente SNMP de cada uno y guarda las métricas.
4. La web muestra el estado actual y el histórico.

5. Modelo de datos

Tabla	Columnas	Para qué sirve
servers	`id, hostname, ip, comunidad, enabled`	Inventario de servidores a monitorear. *enabled* permite pausar uno sin borrarlo.
metrics	`id, server_id, fecha, cpu, ram, disco, uptime`	Una fila por cada lectura exitosa. Es la tabla que más crece: la web la usa para los gráficos.
settings	`clave, valor`	Configuración del monitoreo (intervalo, umbrales, community por defecto, retención) y el latido del worker, que la web usa para avisar si dejó de funcionar.

Relación: un servidor tiene muchas métricas (`metrics.server_id` apunta a `servers.id`). Todas las fechas se guardan en UTC.

6. Tecnologías

Tecnología	Rol en el proyecto
Docker y Docker Compose	Empaquetan y levantan los tres servicios con un solo comando.
Python + PySNMP	Lógica del worker: consultas SNMP y procesamiento de datos.
MySQL 8	Persistencia de inventario, métricas y configuración.
Flask (interfaz web)	Servidor web y API que lee y escribe en MySQL.
Bootstrap y ECharts.js	Diseño de la interfaz y gráficos históricos en el navegador.

7. Qué incluye la implementación

Los tres módulos viven en una sola carpeta (`worker/`, `mysql/`, `web/`), cada uno en su contenedor, y un único script (`./start.sh`) levanta todo el servidor controlador. Los servidores Ubuntu solo necesitan `snmpd`; el proyecto incluye un script para configurarlos. La web tiene además un **modo demo** con datos de ejemplo, útil para mostrarla sin servidores.

- **Panel:** resumen de disponibilidad, tarjetas por servidor con CPU, RAM, disco y uptime, y gráfico de las últimas 24 horas.
- **Servidores:** ABM completo (agregar, editar, habilitar o deshabilitar y eliminar).
- **Histórico:** gráficos por servidor y período, estadísticas (promedio, mínimo y máximo) y tabla de últimas mediciones.
- **Configuración:** intervalo de consulta, umbrales de aviso y críticos, community por defecto y retención.
- **Aviso del worker:** el Panel indica si el worker dejó de reportar, para no confundirlo con servidores caídos.

8. Mejoras posibles

1. Probar con servidores Ubuntu reales (o máquinas virtuales) y validar los OIDs con snmpget o snmpwalk.
2. Mostrar en la web el motivo por el que un servidor no responde (timeout, community incorrecta, OID faltante).
3. Alertas por correo cuando un servidor supera el umbral crítico o deja de responder.
4. Migrar a SNMPv3 (autenticación y cifrado) y poner la web detrás de un proxy con HTTPS.